

Gabarito — Lista de exercícios — Grafos

Convenções usadas no gabarito (VISTAS EM SALA):

- Na BFS, a ordem de visitação é a ordem de **desenfileamento**.
- Na DFS iterativa, a ordem de visitação é a ordem de **desempilhamento** e os vizinhos são marcados como descobertos no momento em que são **empilhados**.
- Nas tabelas de Dijkstra, cada entrada aparece no formato (*distancia, predecessor*).

1. Grafo:

$$V = \{E, R, A, B, C, D\}$$

$$\text{Arestas} = \{E-R, R-A, A-B, B-R, R-C, C-D, D-R\}$$

Graus:

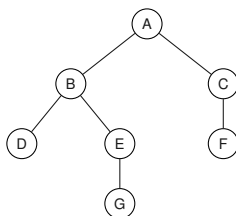
$$g(E) = 1, \quad g(R) = 5, \quad g(A) = 2, \\ g(B) = 2, \quad g(C) = 2, \quad g(D) = 2.$$

Não é possível fazer um circuito euleriano iniciando e terminando em *E*, pois existem vértices de grau ímpar. Para existir circuito euleriano, todos os vértices devem ter grau par.

2. (a) Ordem de visitação na BFS:

A, B, C, D, E, F, G

(b) Árvore de busca:

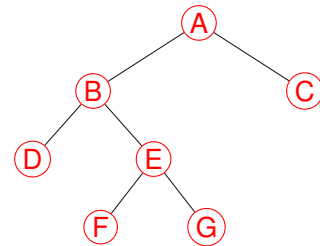


(c) Caminho encontrado:

A → B → E → G

3. (a) A ordem de visitação é a ordem em que os vértices são efetivamente visitados/desempilhados, e para garantir menor valor primeiro na pilha, os vizinhos devem ser empilhados em ordem inversa.
Ordem de visitação na DFS com pilha: *A, B, D, E, F, G, C*

(b) Árvore de busca:



4. Ordem dos vértices na tabela: *A, B, C, D, E, F, G, H*.

(a) Distância mínima entre *B* e *E*: **6**

(b) Caminho encontrado: *B → D → G → E*

(c) Atualização das distâncias e predecessores:

Etapa	A	B	C	D	E	F	G	H
Início	(∞, -)	(0, -)	(∞, -)	(∞, -)	(∞, -)	(∞, -)	(∞, -)	(∞, -)
1ª it (B)	(1, B)	(0, -)	(∞, -)	(2, B)	(∞, -)	(∞, -)	(4, B)	(6, B)
2ª it (A)	(1, B)	(0, -)	(9, A)	(2, B)	(∞, -)	(∞, -)	(4, B)	(5, A)
3ª it (D)	(1, B)	(0, -)	(9, A)	(2, B)	(∞, -)	(4, D)	(3, D)	(5, A)
4ª it (G)	(1, B)	(0, -)	(9, A)	(2, B)	(6, G)	(4, D)	(3, D)	(5, A)
5ª it (F)	(1, B)	(0, -)	(5, F)	(2, B)	(6, G)	(4, D)	(3, D)	(5, A)
Final	(1, B)	(0, -)	(5, F)	(2, B)	(6, G)	(4, D)	(3, D)	(5, A)

Vetor final de distâncias: {1, 0, 5, 2, 6, 4, 3, 5}

5. Uma possível solução é:

```

void dfs_aux(Grafo *g, int atual, int *visitado) {
    visitado[atual] = 1;
    printf("%d ", g->v[atual]);

    for (int i = 0; i < g->n; i++) {
        if (g->M[atual][i] != g->arestanula &&
            !visitado[i]) {
            dfs_aux(g, i, visitado);
        }
    }
}

void dfs_recursiva(Grafo *g, int origem) {
    int *visitado = calloc(g->n, sizeof(int));

    dfs_aux(g, origem, visitado);

    free(visitado);
}
  
```

6. Uma possível solução usando fila dinâmica:

```
int existe_caminho_bfs(Grafo *g,
                      int origem,
                      int destino) {
    int *descoberto = calloc(g->n, sizeof(int));

    Fila *fila = criarFila();

    enfileirar(fila, origem);
    descoberto[origem] = 1;

    while (!filaVazia(fila)) {
        int atual;
        desenfileirar(fila, &atual);

        if (atual == destino) {
            free(descoberto);
            destruirFila(fila);
            return 1;
        }

        for (int i = 0; i < g->n; i++) {
            if (g->M[atual][i] != g->arestanula &&
                !descoberto[i]) {
                enfileirar(fila, i);
                descoberto[i] = 1;
            }
        }
    }

    free(descoberto);
    destruirFila(fila);
    return 0;
}
```

7. Uma possível solução é:

```
int dfs_contar(Grafo *g, int atual, int *descoberto) {
    descoberto[atual] = 1;
    int total = 1;

    for (int i = 0; i < g->n; i++) {
        if (g->M[atual][i] != g->arestanula && !
            descoberto[i]) {
            total += dfs_contar(g, i, descoberto);
        }
    }

    return total;
}

int contar_alcancaveis(Grafo *g, int origem) {
    int *descoberto = calloc(g->n, sizeof(int));

    int total = dfs_contar(g, origem, descoberto);

    free(descoberto);

    return total;
}
```

8. Uma possível solução é:

```
int* obter_adjacentes(Grafo *g,
                     int vertice,
                     int *quantidade) {
    *quantidade = 0;

    for (int i = 0; i < g->n; i++) {
        if (g->M[vertice][i] != g->arestanula) {
            (*quantidade)++;
        }
    }

    int *adj = malloc((*quantidade) * sizeof(int));

    int pos = 0;
    for (int i = 0; i < g->n; i++) {
        if (g->M[vertice][i] != g->arestanula) {
            adj[pos++] = g->v[i];
        }
    }

    return adj;
}
```

9. Uma possível solução é:

```
int eh_isolado(Grafo *g, int vertice) {
    for (int i = 0; i < g->n; i++) {
        if (g->M[vertice][i] != g->arestanula) {
            return 0;
        }
    }

    return 1;
}
```

10. Uma possível solução é:

```
int contar_arestas(Grafo *g) {
    int total = 0;

    for (int i = 0; i < g->n; i++) {
        for (int j = i + 1; j < g->n; j++) {
            if (g->M[i][j] != g->arestanula) {
                total++;
            }
        }
    }

    return total;
}
```

11. Uma possível solução é:

```
int grau_entrada(Grafo *g, int vertice) {
    int grau = 0;

    for (int i = 0; i < g->n; i++) {
        if (g->M[i][vertice] != g->arestanula) {
            grau++;
        }
    }

    return grau;
}
```

12. Uma possível solução é:

```
int grau_saida(Grafo *g, int vertice) {
    int grau = 0;

    for (int j = 0; j < g->n; j++) {
        if (g->M[vertice][j] != g->arestanula) {
            grau++;
        }
    }

    return grau;
}
```

13. Uma possível solução é:

```
void marcar_componente(Grafo *g,
                      int atual,
                      int *visitado) {
    visitado[atual] = 1;

    for (int i = 0; i < g->n; i++) {
        if (g->M[atual][i] != g->arestanula &&
            !visitado[i]) {
            marcar_componente(g, i, visitado);
        }
    }
}

int contar_componentes(Grafo *g) {
    int *visitado = calloc(g->n, sizeof(int));
    int componentes = 0;

    for (int i = 0; i < g->n; i++) {
        if (!visitado[i]) {
            componentes++;
            marcar_componente(g, i, visitado);
        }
    }

    free(visitado);
    return componentes;
}
```

14. Grafo completo não direcionado:

$$\frac{n(n-1)}{2}$$

Grafo orientado completo:

$$n(n-1)$$

15. Graus:

$$g(P) = 3, \quad g(R) = 4, \quad g(S1) = 3, \quad g(S2) = 3,$$

$$g(S3) = 2, \quad g(S4) = 3, \quad g(S5) = 3, \quad g(S6) = 3.$$

Não existe circuito euleriano, pois há vértices de grau ímpar. Para existir circuito euleriano, todos os vértices deveriam ter grau par.

16. Grafo G_1 :

(a) Não direcionado.

(b) Não ponderado.

(c) Simples.

(d) Conexo.

(e) Não completo.

(f) Não regular.

(g) Possui ciclo, por exemplo:

$$A - B - D - C - A$$

Grafo G_2 :

(a) Direcionado.

(b) Não ponderado.

(c) Simples.

(d) Conexo.

(e) Não completo.

(f) Não regular.

(g) Possui ciclo, por exemplo:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$$

17. Ordem dos vértices na tabela: A, B, C, D, E, F, G.

Etapa	A	B	C	D	E	F	G
Início	(∞ , -)	(∞ , -)	(∞ , -)	(0, -)	(∞ , -)	(∞ , -)	(∞ , -)
1ª it (D)	(5, D)	(9, D)	(∞ , -)	(0, -)	(5, D)	(1, D)	(∞ , -)
2ª it (F)	(5, D)	(9, D)	(∞ , -)	(0, -)	(4, F)	(1, D)	(2, F)
3ª it (G)	(5, D)	(9, D)	(∞ , -)	(0, -)	(3, G)	(1, D)	(2, F)
4ª it (E)	(5, D)	(4, E)	(8, E)	(0, -)	(3, G)	(1, D)	(2, F)

(a) Após a inicialização:

$$\{\infty, \infty, \infty, 0, \infty, \infty, \infty\}$$

(b) Após a 3ª iteração:

$$\{5, 9, \infty, 0, 3, 1, 2\}$$

(c) Após a 4ª iteração:

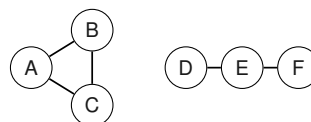
$$\{5, 4, 8, 0, 3, 1, 2\}$$

(d) Vértices explorados:

$$1^a : D \quad 2^a : F$$

18. (a) É um grafo não direcionado, representado por matriz de adjacência.

(b) Grafo correspondente:



(c) Possui 2 componentes conexas.

Possui 1 ciclo:

$$A - B - C - A$$

(d) Lista de adjacência:

```
A: B, C
B: A, C
C: A, B
D: E
E: D, F
F: E
```

19. Existe caminho entre 9 e 12.

Ordem de desenfileamento até encontrar o destino:

$$9, 0, 2, 15, 14, 1, 10, 4, 8, 7, 13, 5, 12$$

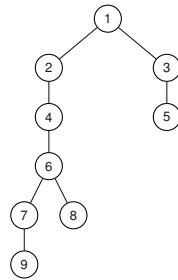
Caminho encontrado:

$$9 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 7 \rightarrow 12$$

20. (a) Ordem de visitação na BFS:

1, 2, 3, 4, 5, 6, 7, 8, 9

(b) Árvore de busca:



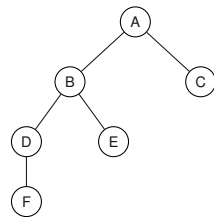
(c) Menor caminho:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 9$

(d) O caminho possui 6 vértices.

21. (a) **Ordem de visitação na DFS com pilha: A ordem de visitação é a ordem em que os vértices são efetivamente visitados/desempilhados, e para garantir menor valor primeiro na pilha, os vizinhos devem ser empilhados em ordem inversa. A, B, D, F, E, C**

(b) Árvore de busca:



(c) Arestas que não pertencem à árvore de busca:

$F \rightarrow E, \quad E \rightarrow C.$

(d) O dígrafo não possui ciclo direcionado.

22. Ordem dos vértices: A, B, C, D, E, F.

Etapas	A	B	C	D	E	F
Início	(0, -)	(∞, -)	(∞, -)	(∞, -)	(∞, -)	(∞, -)
1ª it (A)	(0, -)	(2, A)	(5, A)	(∞, -)	(∞, -)	(∞, -)
2ª it (B)	(0, -)	(2, A)	(3, B)	(5, B)	(∞, -)	(9, B)
3ª it (C)	(0, -)	(2, A)	(3, B)	(5, B)	(5, C)	(9, B)
4ª it (D)	(0, -)	(2, A)	(3, B)	(5, B)	(5, C)	(9, B)
5ª it (E)	(0, -)	(2, A)	(3, B)	(5, B)	(5, C)	(6, E)
6ª it (F)	(0, -)	(2, A)	(3, B)	(5, B)	(5, C)	(6, E)

(a) Após a inicialização:

$\{0, \infty, \infty, \infty, \infty, \infty\}$

(b) Ordem de exploração:

A, B, C, D, E, F

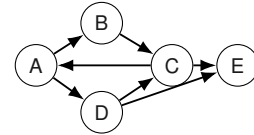
(c) Distância mínima de A até F: 6

(d) Caminho mínimo:

$A \rightarrow B \rightarrow C \rightarrow E \rightarrow F$

23. (a) O grafo é direcionado.

(b) Grafo correspondente:



(c) Grau de saída:

$g_{out}(A) = 2, \quad g_{out}(B) = 1, \quad g_{out}(C) = 2,$

$g_{out}(D) = 2, \quad g_{out}(E) = 0.$

(d) Grau de entrada:

$g_{in}(A) = 1, \quad g_{in}(B) = 1, \quad g_{in}(C) = 2,$

$g_{in}(D) = 1, \quad g_{in}(E) = 2.$

(e) Sim. Um caminho possível é:

$A \rightarrow D \rightarrow E$

24. (a) Não é conexo.

(b) Possui 5 componentes conexas:

$\{A\}, \{B, C\}, \{D, E, F\}, \{G\}, \{H\}.$

(c) Vértices isolados:

A, G, H.

(d) Possui ciclo:

$D - E - F - D.$

(e) Graus:

$g(A) = 0, \quad g(B) = 1, \quad g(C) = 1,$

$g(D) = 2, \quad g(E) = 2, \quad g(F) = 2,$

$g(G) = 0, \quad g(H) = 0.$

25. Ordem dos vértices na matriz:

M, N, O, P, Q, R, S.

(a) Um caminho simples de M até S é:

$M \rightarrow S.$

(b) Não existe ciclo simples com pelo menos 4 nós. Existem ciclos menores, como:

$M \rightarrow S \rightarrow M$

e

$N \rightarrow O \rightarrow N.$

(c) Graus:

$g_{out}(N) = 4, \quad g_{in}(N) = 1, \quad g(N) = 5.$

$g_{out}(R) = 0, \quad g_{in}(R) = 2, \quad g(R) = 2.$

(d) Lista de adjacência:

M: S, S
 N: O, S, Q, R
 O: M, M, N
 P: R, P, M
 Q:
 R:
 S: M, M

(e) Matriz de adjacência:

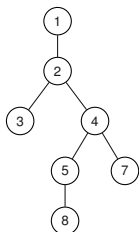
$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 2 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

26. Grafo 1

(a) BFS:

1, 2, 3, 4, 5, 7, 8

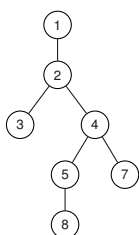
(b) Árvore da BFS:



(c) DFS com pilha:

1, 2, 4, 7, 5, 8, 3

(d) Árvore da DFS:

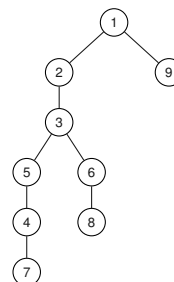


Grafo 2

(a) BFS:

1, 2, 9, 3, 5, 6, 4, 8, 7

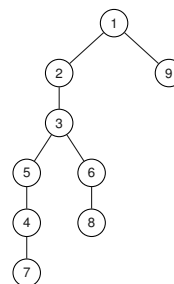
(b) Árvore da BFS:



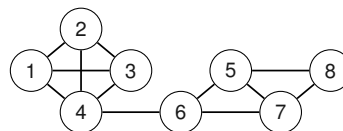
(c) DFS com pilha:

1, 9, 2, 3, 6, 8, 5, 4, 7

(d) Árvore da DFS:



27. (a) Grafo:



(b) Matriz de adjacência:

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

(c) Lista de adjacência:

1: 2, 3, 4
 2: 1, 3, 4
 3: 1, 2, 4
 4: 1, 2, 3, 6
 5: 6, 7, 8
 6: 4, 5, 7
 7: 5, 6, 8
 8: 5, 7

28. Considerando a ordem em que os vizinhos aparecem nas listas:

BFS a partir de A :

A, E, F, B, G, C, D, H

DFS com pilha a partir de A :

A, B, F, C, D, H, G, E

Distâncias até A , usando BFS:

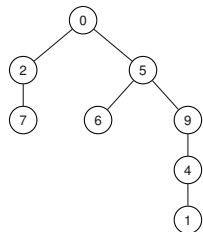
$$d(A) = 0, \quad d(E) = 1, \quad d(F) = 1, \quad d(B) = 1,$$

$$d(G) = 2, \quad d(C) = 2, \quad d(D) = 3, \quad d(H) = 4.$$

29. (a) BFS:

0, 2, 5, 7, 6, 9, 4, 1

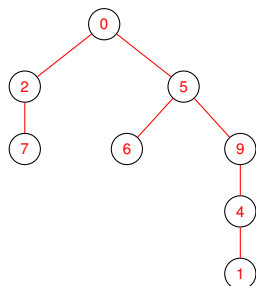
(b) Árvore da BFS:



(c) DFS com pilha: A ordem de visitação é a ordem em que os vértices são efetivamente visitados/desempilhados, e para garantir menor valor primeiro na pilha, os vizinhos devem ser empilhados em ordem inversa.

Ordem de visitação na DFS com pilha: 0, 2, 7, 5, 6, 9, 4, 1

(d) Árvore da DFS:



30. Ordem dos vértices:

A, B, C, D, E, F, G .

Origem A :

Etapa	A	B	C	D	E	F	G
Final	$(0, -)$	$(5, A)$	$(3, A)$	$(9, E)$	$(7, G)$	$(8, E)$	$(6, B)$

(a) Vetor de distâncias:

$\{0, 5, 3, 9, 7, 8, 6\}$

(b) Vetor de predecessores:

$\{-, A, A, E, G, E, B\}$

(c) Caminhos mínimos:

$A : A$

$B : A \rightarrow B$

$C : A \rightarrow C$

$G : A \rightarrow B \rightarrow G$

$E : A \rightarrow B \rightarrow G \rightarrow E$

$F : A \rightarrow B \rightarrow G \rightarrow E \rightarrow F$

$D : A \rightarrow B \rightarrow G \rightarrow E \rightarrow D$

Origem B :

Etapa	A	B	C	D	E	F	G
Final	$(6, D)$	$(0, -)$	$(2, B)$	$(4, E)$	$(2, G)$	$(3, E)$	$(1, B)$

(a) Vetor de distâncias:

$\{6, 0, 2, 4, 2, 3, 1\}$

(b) Vetor de predecessores:

$\{D, -, B, E, G, E, B\}$

$A : D, B : -, C : B, D : E, E : G, F : E, G : B$.

(c) Caminhos mínimos:

$A : B \rightarrow G \rightarrow E \rightarrow D \rightarrow A$

$B : B$

$C : B \rightarrow C$

$G : B \rightarrow G$

$E : B \rightarrow G \rightarrow E$

$F : B \rightarrow G \rightarrow E \rightarrow F$

$D : B \rightarrow G \rightarrow E \rightarrow D$

31. Ordem dos vértices:

$Jensen, Leuzingner, Dufour, Medeiros$.

(a) Vértices:

$V = \{Jensen, Leuzingner, Dufour, Medeiros\}$.

(b) Arestas direcionadas:

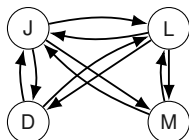
$J \rightarrow L, \quad J \rightarrow D, \quad J \rightarrow M,$

$L \rightarrow J, \quad L \rightarrow D, \quad L \rightarrow M,$

$D \rightarrow J, \quad D \rightarrow L,$

$M \rightarrow J, \quad M \rightarrow L.$

(c) Dígrafo:



(d) Lista de adjacência:

Jensen: Leuzingner, Dufour, Medeiros
 Leuzingner: Jensen, Dufour, Medeiros
 Dufour: Jensen, Leuzingner
 Medeiros: Jensen, Leuzingner

(e) Matriz de adjacência:

$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

32. Ordem dos vértices:

0, 1, 2, 3, 4, 5.

Etapas	0	1	2	3	4	5
Início	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(\infty, -)$	$(0, -)$
1ª it (5)	(2, 5)	(4, 5)	$(\infty, -)$	(6, 5)	$(\infty, -)$	(0, -)
2ª it (0)	(2, 5)	(4, 5)	$(\infty, -)$	(6, 5)	(17, 0)	(0, -)
3ª it (1)	(2, 5)	(4, 5)	$(\infty, -)$	(6, 5)	(17, 0)	(0, -)
4ª it (3)	(2, 5)	(4, 5)	(18, 3)	(6, 5)	(17, 0)	(0, -)
5ª it (4)	(2, 5)	(4, 5)	(18, 3)	(6, 5)	(17, 0)	(0, -)
6ª it (2)	(2, 5)	(4, 5)	(18, 3)	(6, 5)	(17, 0)	(0, -)

(a) Menor caminho encontrado:

5 → 3 → 2

(b) Custo total:

18

33. Grafo (a).

Matriz de adjacência na ordem (1, 2, 3, 4, 5):

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 2 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 2 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Lista de adjacência:

1: 4, 5, 5
 2: 2, 3, 4, 5
 3: 2, 4
 4: 1, 2, 3
 5: 1, 1, 2

Grafo (b).

Matriz de adjacência na ordem (1, 2, 3, 4, 5, 6, 7):

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

Lista de adjacência:

1: 2, 3
 2: 1, 3, 4, 6, 7
 3: 1, 2, 5, 6, 7
 4: 2, 6
 5: 3, 7
 6: 2, 3, 4, 7
 7: 2, 3, 5, 6

Grafo (c).

Matriz de adjacência na ordem (1, 2, 3, 4):

$$\begin{bmatrix} 0 & 3 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Lista de adjacência:

1: 2, 2, 2, 3
 2: 2, 3, 4
 3: 4
 4: